cloudtorch / develop — Step 6 outline

Deploy the validated step-1–5 stack as a single configurable, resumable command-line script that fits the *full* (t, y, x, λ) cube on GPU: notebook→script, config/CLI, `device=cuda`, coordinate minibatching, checkpointing, saved parameter maps. Deliverable of `instructions.md` step 6 — the final step.

---

1. OBJECTIVE

Turn the validated stack — today the notebook `step_5_reg_test.ipynb` — into one configurable, resumable script that fits the *full* (t, y, x, λ) cube on GPU. **No new physics or model changes:** `ParameterField`, `composite_synth`, `chi2_per_pixel` and `regularization_loss` are reused verbatim; only the driver (data → basis → field → train → save) is new. Success: a no-arg run reproduces the step-5 notebook, and the full-field run is reached by editing *config alone*. This closes `instructions.md`.

2. NOTEBOOK → SCRIPT

A thin driver (`fit_cube.py`) mirrors the notebook cells as functions: `load_data` (auto-detect path, border and λ crop, normalise by I_{qs}), `build_basis` (`fit_background_basis`, or load a persisted basis), `build_field` (`ParameterField` + the field-average `bkg_bias`), `train` (cosine-annealed Adam; loss = χ^2+`regularization_loss`; grad-clip; optional minibatch), `save_outputs`. The diagnostic/plot cells (emission check, maps, spectra, residuals) become an optional `--report` that writes the figures + the χ^2/RMS/emission summary to the out-dir, *off* the training hot path. The per-pixel core (steps 1–5) is untouched.

3. CONFIG & CLI

One grouped config object (dataclass or dict) exposing every top-of-notebook knob: *data* (path list, border `YS/XS`, `LAM`, `N_BASIS`, `K`, frames `T0/N_T` or *all*, `CROP` or *full*); *field* (`N_FREQ`, `SIGMA_XY`, `SIGMA_T`, `WIDTH`, `DEPTH`, `S_MAX`, `DV_FLOOR`, `seed`); *train* (`N_ITERS`, `LR`, `GRAD_CLIP`, `BATCH`, `device`); and the grouped `REG` dict verbatim. Loaded from a JSON/YAML file, with a few `argparse` overrides (`--config`, `--resume`, `--out`, `--device`, `--crop`, `--n-t`) for quick scans. The step-5 values *are* the default config, so weight/scale scans are just re-launches with a different file.

4. FULL-CUBE COORDINATES & SCALE TRANSFER

`grid_coords` normalises *each* axis to $[-1, 1]$ over its own extent, so `SIGMA_XY/SIGMA_T` and the smoothness weights w^{xy}, w^t live in *normalised* units — their physical meaning (cycles per pixel; smoothing length) drifts with crop and frame count. Enlarging the config naively would silently re-scale the step-4/5 tuning. **Fix:** normalise to a *fixed physical scale* — 1 unit = `REF_XY` pixels / `REF_T` frames (config-set), centred on the cube — *not* to the per-domain $[-1, 1]$. Choose `REF` so the $40 \times 40 \times 4$ tuning crop reproduces today's $[-1, 1]$; then the tuned σ and weights transfer to the full field unchanged (it merely spans a wider coordinate range at the same per-pixel scale). This is a small addition to `grid_coords` (a physical-scale option); the fallback, if no code change is wanted, is to rescale $\sigma \propto n$ and $w \propto 1/n^2$ by hand per run. Because every loss term is a per-pixel *mean*, the `REG` weights are invariant to `BATCH` and pixel-count — only this normalisation needs care.

5. CHECKPOINTING, RESUME & OUTPUTS

Periodic + final checkpoints hold `field.state_dict`, optimiser, scheduler, iteration, and the RNG states (torch/cuda — for reproducible minibatching), plus the config and the PCA basis (`pca.save`), so a checkpoint is self-contained. `--resume` restores all and continues the cosine schedule. Final output: evaluate the field over the full grid under `no_grad`, reshape to $(N_t, N_y, N_x, \cdot)$, and save an `.npz` of the background coeffs `c` $(\cdot, 6)$ and `p_cloud` $(\cdot, 10)$ (incl. the frozen `loga`) with metadata (`wav`, `LAM`, I_{qs}, config); optional FITS via `utils.writefits`. Determinism via the config seeds; the `--report` block re-uses the notebook's diagnostics for QC.

Plan only — no code until you approve. Then `fit_cube.py` + a default config (the step-5 values) + a small `grid_coords` physical-scale option; steps 1–5 stay unchanged. **Final step** — closes `instructions.md`: the full (t, y, x, λ) cube fit on GPU, configurable, resumable, with saved parameter maps.